# Bridging the Enforcement Verification Gap: Detecting Divergence Between Declared and Actual Security Posture in Kubernetes Containerized Environments

Yehia Sobeh
Innopolis University, Innopolis, Russia
yehiasobehacm@gmail.com

Ruslan Damindarov
Innopolis University, Innopolis, Russia
Rubetek AI and Robotics Lab
r.damindarov@innopolis.university

Awwal Ishiaku
Innopolis University, Innopolis, Russia
a.ishiaku@innopolis.university

***Abstract*—Kubernetes secures containerized workloads through declarative security primitives, assuming that the container runtime faithfully translates declared intent into kernel-level enforcement. This paper identifies and addresses the *enforcement verification gap*: the absence of tooling that verifies whether declared Kubernetes security contexts are actually enforced by the container runtime. We present k8s-psa, a DaemonSet-based auditor that continuously compares pod security specifications from the Kubernetes API against the actual runtime state obtained through the container runtime interface. In controlled experiments on a Minikube cluster with the Docker runtime, we evaluated the tool across seven test scenarios. The results reveal two findings: privileged containers that declare seccomp filtering have it silently stripped by the Docker runtime (a genuine enforcement gap invisible to specification-only tools), and the Docker metadata interface omits the seccomp status of correctly protected non-privileged containers (an observability gap producing false positives in naive runtime inspection). A kernel-verified validation on containerd shows that the enforcement gap generalizes across runtimes, whereas the observability gap is Docker-specific. These findings demonstrate that the enforcement verification gap is real and that runtime inspection must account for the semantic differences between runtime metadata and actual kernel enforcement.**

***Index Terms*—Kubernetes, container security, seccomp, runtime verification, enforcement drift, DevSecOps, security context**

## I. Introduction

Modern software systems increasingly adopt microservice architectures deployed as containerized workloads on Kubernetes. This shift introduces security challenges that traditional perimeter-based approaches cannot address: a larger attack surface distributed across potentially hundreds of containers, complex dependency chains that pull in third-party images, and dynamic runtime behavior [1], [2].

Kubernetes addresses these challenges through declarative security primitives specified in pod security contexts. Admission controllers such as OPA Gatekeeper [3] and Kyverno [4] intercept pod creation requests and reject non-compliant specifications, while Pod Security Admission (PSA), which has been stable since Kubernetes v1.25 [5], enforces predefined security profiles at the namespace level.

Despite their differences, these tools share a critical limitation: every one of them reads security properties from the Kubernetes API, that is, from the *declared* pod specification. None of them verifies that the container runtime has correctly translated those declarations into kernel-level enforcement. This verification matters because the translation from a pod specification to actual process isolation passes through multiple layers (kubelet to Container Runtime Interface (CRI) request, container runtime to Open Container Initiative (OCI) specification, low-level runtime to kernel primitives [6]), and each step may add defaults, override values, or fail to apply certain restrictions.

This paper terms the resulting blind spot the *enforcement verification gap* and presents k8s-psa,[1] a tool designed to detect divergences between declared security intent and actual runtime enforcement.

### A. Research Questions

Three research questions guide this work. **RQ1:** To what extent does the actual security configuration of running containers diverge from the security properties declared in their Kubernetes pod specifications? **RQ2:** Can a lightweight, DaemonSet-based tool effectively detect such divergences with acceptable accuracy and performance overhead? **RQ3:** What categories of divergence occur in practice, and how should each category be classified from a security perspective?

### B. Contributions

This work makes four contributions: (1) a novel verification tool that closes the enforcement verification gap by systematically comparing declared Kubernetes security contexts against observed container runtime state; (2) kernel-verified

[1]Kubernetes Pod Security Auditor; not to be confused with the built-in Pod Security Admission (PSA) controller, which operates only on declared specifications.

evidence that both Docker and containerd strip declared seccomp profiles from privileged containers without warning; (3) the discovery of a Docker-specific observability gap in runtime metadata; and (4) an open-source tool and evaluation framework.

## II. Background

### A. Kubernetes Security Architecture

Kubernetes enforces workload security through a layered model: pod specifications carry `securityContext` fields defining the intended posture [6], the container runtime turns the kubelet's CRI request into an OCI runtime specification [7] describing the exact kernel configuration, and a low-level runtime configures namespaces, cgroups, capabilities, and seccomp filters.

### B. Pod Security Admission, Policy Engines, and Runtime Monitors

Pod Security Admission, which has been stable since Kubernetes v1.25 [5], enforces Pod Security Standards at admission time but does not monitor running workloads or verify runtime enforcement. Policy engines extend this admission-time model with periodic re-evaluation: OPA Gatekeeper [3] audits cluster resources every 60 seconds, and Kyverno [4] performs background scans every hour. In both cases, however, the audit reads the declared pod specification from the Kubernetes API, and neither tool queries the container runtime.

At the opposite end of the stack, behavioral monitors such as Falco [8] instrument the kernel via eBPF to detect anomalous system calls; they observe what containers *do* at the kernel level, but do not verify whether the active security configuration matches what was declared.

## III. Related Work

### A. Static Misconfiguration Detection

Research on Kubernetes security configuration has concentrated on the specification level. Rahman et al. [10] conducted a large-scale empirical study of 2,039 open-source Kubernetes manifests and explicitly identified runtime verification as future work. Building on the same specification-level view, Malul et al. [11] proposed GenKubeSec for language-model-based YAML misconfiguration detection, and Sgan Cohen et al. [12] presented KubeGuard for RBAC and NetworkPolicy hardening; both works, which have not yet undergone peer review, likewise operate on declared configuration rather than enforced state.

### B. Configuration Drift Detection

A second line of work targets configuration drift, exemplified by Kubediff [13], which reports differences between the running Kubernetes state and version-controlled manifests. Approaches in this category treat the Kubernetes API as the source of truth for the running state; none extends below the API to the container runtime layer.

### C. The Identified Gap

Several results indicate that the layer below the API deserves scrutiny. Kermabon-Bobinnec et al. [14] demonstrated that temporal inconsistency between the orchestration and runtime layers can be exploited to bypass security policies, and Shamim et al. [15] concluded that security researchers should investigate dynamic configuration violation detection. Survey work by Wong et al. [1] and Devi Priya et al. [2] likewise identified runtime configuration drift detection as under-explored. To the best of our knowledge, however, no existing open-source tool systematically compares the security declarations held by the Kubernetes API against the state reported by the container runtime. The present work addresses precisely this gap.

## IV. Methodology

### A. System Architecture

k8s-psa operates as a Kubernetes DaemonSet, deploying one scanner pod per cluster node; this guarantees complete node coverage, co-locates each scanner with the local container runtime socket, and scales automatically as nodes join or leave the cluster. Each instance performs periodic two-layer security scans.

**Layer 1 (Spec Check)** reads pod specifications from the Kubernetes API and checks them against eight configurable security policies (privileged mode, root user, host PID/network/IPC, seccomp profile, dangerous capabilities, and read-only root filesystem). This layer serves two purposes: it provides a baseline against which Layer 2 findings can be contextualized, and it flags pods that lack security declarations entirely, a case Layer 2 cannot detect, because the absence of a declaration leaves no intent to verify.

**Layer 2 (Drift Detection)**, the novel contribution, compares declared specifications against actual runtime state by: (1) extracting a `SpecSecurityState` from the Kubernetes API; (2) querying the container runtime (`docker inspect` or `crictl inspect`) to build a `RuntimeSecurityState`; (3) normalizing both states to account for expected divergences; and (4) comparing the normalized states field by field across seven security dimensions.

The comparison engine classifies each divergence as **CRITICAL** (a declared restriction is absent at runtime), **WARNING** (an undeclared capability is present), or **INFO** (the runtime is more restrictive than declared).

### B. Capability Normalization

Because the container runtime adds 14 default Linux capabilities to every non-privileged container [9], naive comparison would flood the operator with false positives. The normalizer therefore computes the *expected* capability set from the pod specification (defaults, minus `drop`, plus `add`); only deviations from this expected set generate findings.

### C. Experimental Environment

Experiments were conducted on a single-node Minikube cluster (v1.35.0) running Kubernetes v1.32.0 with Docker 27.4.1, hosted on an Intel Core i5-1135G7 laptop (4 cores, 8 threads, 8 GB RAM, Ubuntu 22.04.5); the Minikube node was allocated 2 vCPUs and 2200 MB of memory. The scanner interval was 60 seconds. Seven test pods were designed to cover the spectrum of security configurations: fully hardened, no security context, privileged with and without a declared seccomp profile, partially secured, one with privilege escalation restricted, and one sharing host namespaces. Three scan cycles were captured per pod. The scanner additionally observed the Kubernetes control plane pods running in the cluster.

## V. Results

### A. Detection Results Summary

Table I presents the Layer 2 drift findings for each test pod. Layer 1 behaved as expected across all seven pods (e.g., zero violations for the fully hardened Pod 1 and six violations for the host-namespace Pod 7) and is not discussed further; the remainder of this section focuses on the Layer 2 findings that constitute the contribution of this work.

TABLE I
Layer 2 Drift Findings by Test Pod

| Test Pod | CRIT | WARN | INFO | Match |
|---|---|---|---|---|
| 1. Fully Hardened | 1* | 0 | 0 | §V-C |
| 2. No Security Ctx | 0 | 0 | 0 | Yes |
| 3. Priv + Seccomp | 1 | 27 | 0 | Yes |
| 4. Priv, No Seccomp | 0 | 27 | 0 | Yes |
| 5. Partial Security | 1* | 0 | 0 | §V-C |
| 6. No Privesc | 1* | 0 | 0 | §V-C |
| 7. Host Namespaces | 0 | 0 | 2 | Yes |

*Observability false positives analyzed in Section V-C.

### B. True Positive: Privileged Seccomp Stripping

The central finding concerns Test Pod 3, which declares both `privileged: true` and `seccompProfile.type: RuntimeDefault`. For such containers, the Docker runtime grants all 41 Linux capabilities and removes the seccomp filter entirely, even though seccomp filtering is explicitly declared. This behavior is documented in the Moby project issue tracker [16]. The tool reported a CRITICAL `seccomp_mismatch` finding: the specification declares seccomp type `RuntimeDefault`, but the runtime has no seccomp filter (unconfined).

Test Pod 4 serves as the experimental control: it has the same runtime state as Pod 3 but no seccomp declaration, and it produced zero CRITICAL findings. The 27 WARNING-level capability findings are identical between Pods 3 and 4, reflecting the capabilities that privileged mode grants beyond the 14 defaults.

The security implication is direct: an operator who declares `RuntimeDefault` on a privileged container reasonably expects system call filtering to be active, and any specification-only tool reads the Kubernetes API and reports the container as filtered. Only by querying the container runtime does this removal become visible.

### C. Observability Gap

Test Pods 1, 5, and 6, all non-privileged and all declaring `RuntimeDefault`, unexpectedly produced CRITICAL seccomp findings. Investigation revealed that these are false positives caused by a documented but widely misunderstood behavior of the Docker metadata interface. When cri-dockerd translates `RuntimeDefault`, it passes no explicit `SecurityOpt` entry to Docker and instead relies on Docker's implicit default profile, which restricts several dozen sensitive system calls [9]. Because the implicit profile never appears in `docker inspect` output, a correctly protected container is indistinguishable at the metadata level from an unprotected one. The CIS Docker Benchmark (check 5.21) documents exactly this semantics: an empty `SecurityOpt` field indicates that the default profile is *active*, not that seccomp is disabled [17]. The Kubernetes control plane pods exhibited the same phenomenon, confirming the gap is systematic.

### D. Pod 3 vs. Pod 4: Controlled Comparison

Table II presents the controlled comparison between Pod 3 and Pod 4, which isolates the seccomp declaration as the single independent variable.

TABLE II
Controlled Comparison: Pod 3 vs. Pod 4

| Property | Pod 3 | Pod 4 |
|---|---|---|
| Spec: privileged | true | true |
| Spec: seccompProfile | RuntimeDefault | Not declared |
| Runtime: capabilities | All 41 | All 41 |
| Runtime: seccomp | Unconfined | Unconfined |
| L2 CRITICAL | **1** | **0** |
| L2 WARNING | 27 | 27 |

Both pods have identical runtime states; the only difference is the seccomp declaration. The drift detector flags Pod 3 and remains silent on Pod 4, demonstrating its *specificity*: the tool distinguishes "restriction declared but not enforced" from "restriction never declared."

### E. Cross-Runtime Validation on containerd

To test whether these findings are Docker-specific, we redeployed Pods 1, 3, and 4 on a second cluster with the identical Kubernetes version but the containerd runtime (1.7.24, runc 1.2.2), using the kernel's `/proc/<pid>/status` as ground truth. The enforcement gap *generalizes*: Pod 3's declared profile is passed by the kubelet through the CRI but stripped by containerd (kernel `Seccomp: 0`, all 41 capability bits set). The observability gap is *Docker-specific*: `crictl inspect` exposes the full OCI specification, in which Pod 1's seccomp allowlist is visible and matches the kernel state (`Seccomp: 2`).

### F. Performance Characteristics

On the hardware described above, each scan cycle completed in approximately 3.1 seconds for 16 pods, well within the 60-second scan interval. The scanner consumed 64 MiB of memory and fewer than 100m CPU. The tool ships with 44 automated unit tests that execute in 0.2 seconds.

## VI. Discussion

### A. Answers to the Research Questions

The results answer all three research questions. Divergence between declared and enforced security is not hypothetical (RQ1): privileged containers with seccomp declarations exhibit a real enforcement gap, and privileged mode inflates the capability set by 27 capabilities beyond the declared expectation. A DaemonSet-based tool can detect such divergences with modest resources (RQ2): 64 MiB of memory, $<$100m CPU, and 3.1 s per cycle, with the controlled Pod 3/Pod 4 comparison showing that detection is specific rather than merely sensitive. Three categories of divergence emerged (RQ3): a *genuine enforcement gap* (CRITICAL), *capability inflation* (WARNING), and an *observability gap* (false-positive CRITICAL).

The gap is therefore real but nuanced: the enforcement gap creates a false sense of security, and the cross-runtime validation shows it is systemic rather than a quirk of one runtime. The observability gap demonstrates that runtime metadata is a lossy abstraction over kernel state: a definitive verdict requires the kernel's own enforcement status, the ground truth used in Section V-E. k8s-psa thus occupies a previously empty space between specification-level tools, which never see enforcement, and kernel-level monitors such as Falco, which never see intent.

### B. Limitations

Four limitations bound the generality of these results: the full tool evaluation ran on Docker (the containerd validation covered the three decisive pods, and CRI-O remains untested); the tool's automated seccomp check relies on runtime metadata, with the kernel-level verification of Section V-E not yet integrated into the scanner; the evaluation was conducted on single-node clusters; and seven test pods cover the primary security dimensions but not every configuration.

### C. Future Work

Three directions merit investigation: integrating the `/proc/<pid>/status` kernel check into the scanner itself, which would eliminate the observability gap entirely; extending the cross-runtime study to CRI-O and managed multi-node clusters together with a prevalence study over real-world workloads; and investigating whether mutating admission webhooks introduce further enforcement gaps.

## VII. Conclusion

This paper identified and addressed the enforcement verification gap in Kubernetes security. Through controlled experiments, we demonstrated that privileged containers declaring seccomp filtering had that filtering silently removed by the container runtime, on both Docker and containerd, confirmed at the kernel level. We additionally discovered a Docker-specific observability gap that makes correctly protected containers appear unprotected. The tool, its evaluation framework, and its real-time dashboard are provided as open-source contributions.


## Acknowledgment

The authors gratefully acknowledge the support of the Laboratory for Robotics and Artificial Intelligence and the Institute of Information Security at Innopolis University, together with the Rubetek AI and Robotics Lab.


## References


[1] A. Y. Wong, E. G. Chekole, M. Ochoa, and J. Zhou, "On the security of containers: Threat modeling, attack analysis, and mitigation strategies," *Comput. Secur.*, vol. 128, art. 103140, May 2023, doi: 10.1016/j.cose.2023.103140.

[2] Devi Priya V S, S. C. Sethuraman, and M. K. Khan, "Container security: Precaution levels, mitigation strategies, and research perspectives," *Comput. Secur.*, vol. 135, art. 103490, Dec. 2023, doi: 10.1016/j.cose.2023.103490.

[3] Open Policy Agent, "Gatekeeper audit," 2024. [Online]. Available: https://open-policy-agent.github.io/gatekeeper/website/docs/audit/.

[4] Kyverno, "Background scans," 2024. [Online]. Available: https://kyverno.io/docs/policy-reports/background/.

[5] Kubernetes, "Kubernetes v1.25: Pod Security Admission controller in stable," Aug. 2022. [Online]. Available: https://kubernetes.io/blog/2022/08/25/pod-security-admission-stable/.

[6] Kubernetes, "Restrict a container's syscalls with seccomp," 2024. [Online]. Available: https://kubernetes.io/docs/tutorials/security/seccomp/.

[7] Open Container Initiative, "OCI runtime specification," GitHub, 2023. [Online]. Available: https://github.com/opencontainers/runtime-spec/blob/master/config.md.

[8] Sysdig, "Container drift detection with Falco," 2024. [Online]. Available: https://www.sysdig.com/blog/container-drift-detection-with-falco.

[9] Docker, "Seccomp security profiles for Docker," Docker Docs, 2024. [Online]. Available: https://docs.docker.com/engine/security/seccomp/.

[10] A. Rahman, S. I. Shamim, D. B. Bose, and R. Pandita, "Security misconfigurations in open source Kubernetes manifests: An empirical study," *ACM Trans. Softw. Eng. Methodol.*, vol. 32, no. 4, art. 99, pp. 1–36, May 2023, doi: 10.1145/3579639.

[11] E. Malul, Y. Meidan, D. Mimran, Y. Elovici, and A. Shabtai, "GenKubeSec: LLM-based Kubernetes misconfiguration detection, localization, reasoning, and remediation," 2024, *arXiv:2405.19954*, doi: 10.48550/arXiv.2405.19954.

[12] O. Sgan Cohen, E. Malul, Y. Meidan, D. Mimran, Y. Elovici, and A. Shabtai, "KubeGuard: LLM-assisted Kubernetes hardening via configuration files and runtime logs analysis," 2025, *arXiv:2509.04191*, doi: 10.48550/arXiv.2509.04191.

[13] Weaveworks, "Kubediff," GitHub, 2020. [Online]. Available: https://github.com/weaveworks/kubediff.

[14] H. Kermabon-Bobinnec *et al.*, "ProSPEC: Proactive security policy enforcement for containers," in *Proc. 12th ACM Conf. Data Appl. Security Privacy (CODASPY '22)*, Baltimore, MD, USA, Apr. 2022, pp. 1–12, doi: 10.1145/3508398.3511515.

[15] S. I. Shamim, H. Hu, and A. Rahman, "Dynamic application security testing for Kubernetes deployment: An experience report from industry," in *Proc. 33rd ACM Int. Conf. Found. Softw. Eng. (FSE '25), Industry Papers Track*, Trondheim, Norway, Jun. 2025, doi: 10.1145/3696630.3728573.

[16] Moby, "Allow for custom seccomp profiles to be used in privileged containers," GitHub, Issue #41321, 2020. [Online]. Available: https://github.com/moby/moby/issues/41321.

[17] Center for Internet Security, "CIS Docker Benchmark," v1.7.0, 2024. [Online]. Available: https://www.cisecurity.org/benchmark/docker.